

Task Creation and Management

Task Creation and Management

1. Task Control Overview
2. Core Concepts
 - 2.1 Task State Machine
 - 2.2 Task Creation API
 - 2.3 Task Control API
 - 2.4 Dual-Core Architecture
3. Project Architecture and Flow
4. Hardware Dependencies
5. Source Code Analysis
 - 5.1 Header Files and Macro Configuration
 - 5.2 ws2812_init() — WS2812 Initialization
 - 5.3 ws2812_write_rgb() — Write GRB Color
 - 5.4 vButtonTask() — Button Detection Task
 - 5.5 vLedTask() — LED Task
 - 5.6 vMainControlTask() — Main Control Task
 - 5.7 app_main() — Main Entry
6. Operation Flow
 - 6.1 Compilation and Flashing Flow
 - 6.2 Normal Operation
7. FAQ

1. Task Control Overview

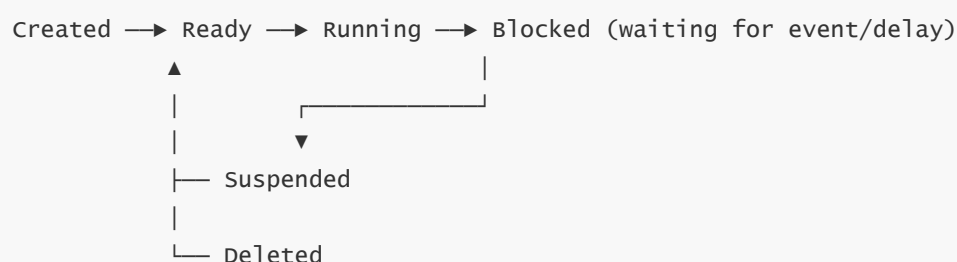
A **FreeRTOS Task** is the **fundamental execution unit** of an embedded real-time operating system. Each task is an independent C function with its own stack space and priority, managed uniformly by the kernel scheduler.

This project demonstrates three types of task lifecycle control: **creation, suspend/resume, and deletion**, driven by a physical button (GPIO0) to switch states, with WS2812 RGB LED displaying task status.

2. Core Concepts

2.1 Task State Machine

FreeRTOS tasks transition between five states from creation to deletion:



State	Meaning
Ready	Ready, waiting for scheduler to allocate CPU
Running	Currently executing (only one task per core at any time)
Blocked	Blocked (waiting for semaphore/queue/delay)
Suspended	Suspended, not participating in scheduling (requires Resume to recover)
Deleted	Deleted, memory reclaimed

2.2 Task Creation API

`xTaskCreate` is the core API for creating tasks. Here are the function signatures for the standard and dual-core pinned versions:

```
// Create task (auto-assign core)
 BaseType_t xTaskCreate(
     TaskFunction_t pvTaskCode,    // Task function pointer
     // Task name (for debugging, max 16 chars)
     const char * const pcName,
     // Stack size in WORDS, not bytes
     uint32_t usStackDepth,
     // Parameter passed to task
     void *pvParameters,
     // Priority (higher number = higher priority)
     UBaseType_t uxPriority,
     // Task handle (NULL if not needed)
     TaskHandle_t *pxCreatedTask
 );

// Pinned version: specify core
 BaseType_t xTaskCreatePinnedToCore(
     ...same as above...,
     BaseType_t xCoreID           // 0 = Core 0 (PRO_CPU), 1 = Core 1 (APP_CPU)
 );
```

ESP32-S3 has 2 cores. `xTaskCreate` automatically assigns cores via the scheduler; `xTaskCreatePinnedToCore` can pin to a specific core. Cross-core access to shared resources still requires mutex protection.

2.3 Task Control API

API	Function	Notes
<code>vTaskSuspend(handle)</code>	Suspend task	Task removed from Ready, no longer scheduled
<code>vTaskResume(handle)</code>	Resume task	Resumes suspended task to Ready state

API	Function	Notes
<code>vTaskDelete(handle)</code>	Delete task	NULL = self-delete; cannot delete suspended tasks
<code>vTaskDelay(ticks)</code>	Delay	Timing starts from call moment, task enters Blocked
<code>uxTaskPriorityGet(handle)</code>	Query priority	NULL = query current task
<code>uxTaskGetStackHighWaterMark(handle)</code>	Stack high water mark	Returns minimum remaining stack space (in words)

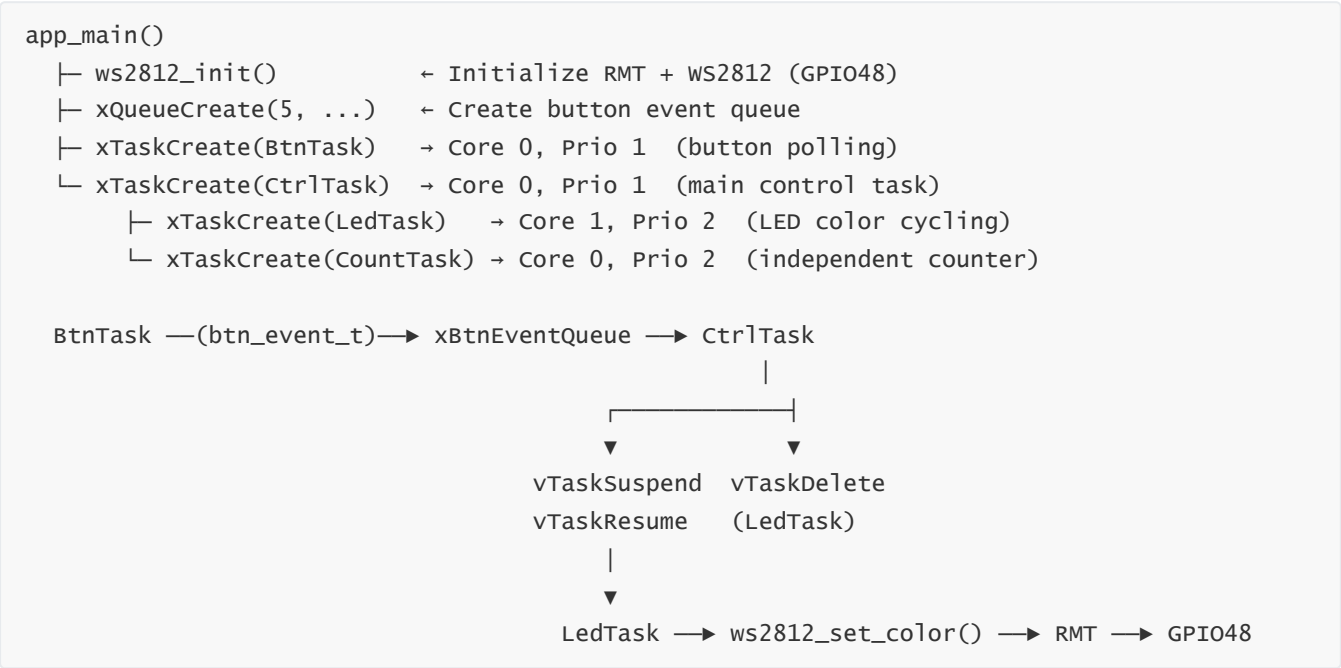
2.4 Dual-Core Architecture

ESP32-S3 FreeRTOS has two cores, each scheduling independently. This project assigns different tasks to different cores:

Core	Purpose
Core 0 (PRO_CPU)	Protocol stack, Wi-Fi, Bluetooth and other system tasks prioritized
Core 1 (APP_CPU)	User application tasks

3. Project Architecture and Flow

The following structure diagram shows the entire project's initialization sequence, task topology, and data flow:



Event	Trigger Condition	Behavior
BTN_EVT_SHORT	30ms ≤ hold < 1000ms	Toggle LedTask suspend/resume
BTN_EVT_LONG	hold ≥ 1000ms	Delete LedTask

Task Topology:

Task	Core	Priority	Stack	Responsibility
BtnTask	0	1	2048	Poll GPIO0, debounce, send button events to queue
CtrlTask	0	1	3072	Receive events from queue, control LedTask suspend/resume/delete
LedTask	1	2	3072	WS2812 color cycling (red→green→blue→white, 1s interval)
CountTask	0	2	2048	Independent counting, print count and stack high water mark every second

4. Hardware Dependencies

Pin	Function	Description
GPIO 0	Button input (KEY0)	Internal pull-up, active low
GPIO 48	WS2812 RGB LED	RMT peripheral driver, 10MHz resolution

Button Debounce Parameters:

Parameter	Value
Polling period	10 ms
Debounce time	30 ms
Long press threshold	1000 ms

WS2812 Timing Parameters (10MHz RMT, 0.1µs/tick):

Parameter	Tick Value	Actual Time
T0H (0-bit high)	4	0.4 µs
T0L (0-bit low)	9	0.9 µs
T1H (1-bit high)	8	0.8 µs
T1L (1-bit low)	5	0.5 µs

Parameter	Tick Value	Actual Time
RESET (reset code)	600	60 μ s (>50 μ s)

Required Components: `freertos/FreeRTOS`, `freertos/task`, `freertos/queue`, `driver/gpio`, `driver/rmt_tx`, `esp_log`

Why not use a third-party RGB LED library? This project directly uses ESP-IDF's built-in RMT peripheral driver (`driver/rmt_tx.h`) to manually encode WS2812 timing, no need to download additional components like `led_strip`. The `ws2812_write_rgb()` function converts RGB to GRB byte order, then builds RMT symbol sequences bit by bit (0.4/0.9 μ s for 0, 0.8/0.5 μ s for 1), with a 60 μ s reset signal at the end, without relying on any third-party library.

5. Source Code Analysis

Full source code: `Source Code` → `ESP-IDF` → `4.FreeRTOS` → `1. RTOS-Task`

5.1 Header Files and Macro Configuration

The following code contains all header files, pin macros, timing parameters, and global handle definitions required by the project:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"           // FreeRTOS kernel
#include "freertos/task.h"               // Task management
#include "freertos/queue.h"              // Queue
#include "driver/gpio.h"                 // GPIO driver
#include "driver/rmt_tx.h"               // RMT TX driver
#include "esp_log.h"                     // Logging library

static const char *TAG = "FREERTOS_TASK"; // Log tag

/* ===== GPIO Pin Definitions ===== */
#define BTN_GPIO 0 // KEY0 button (GPIO 0)
#define RGB_LED_GPIO 48 // WS2812 RGB LED (GPIO 48)

/* ===== WS2812 Timing (10MHz RMT, 0.1us/tick) ===== */
// 10MHz → 0.1us per tick
#define RMT_RESOLUTION_HZ 10000000
// 0-bit high 0.4us
#define WS2812_T0H 4
// 0-bit low 0.9us
#define WS2812_T0L 9
// 1-bit high 0.8us
#define WS2812_T1H 8
// 1-bit low 0.5us
#define WS2812_T1L 5
// Reset 60us (>50us required)
#define WS2812_RESET_TICKS 600

/* ===== Button Timing ===== */
```

```

#define BTN_POLL_PERIOD_MS 10          // Polling period
#define BTN_DEBOUNCE_MS    30          // Debounce time
#define BTN_LONG_PRESS_MS  1000        // Long press threshold

/* ===== Event Definitions ===== */
typedef enum {
    BTN_EVT_SHORT = 0,                // Short press
    BTN_EVT_LONG  = 1,                // Long press
} btn_event_t;

typedef enum {
    LED_COLOR_RED   = 0,              // Red
    LED_COLOR_GREEN = 0,              // Green
    LED_COLOR_BLUE  = 0,              // Blue
    LED_COLOR_WHITE = 0,              // White
    LED_COLOR_OFF   = 0,              // off
} led_color_t;

/* ===== Task & Queue Handles ===== */
static TaskHandle_t xLedTaskHandle   = NULL; // LED task handle
static TaskHandle_t xCountTaskHandle = NULL; // Count task handle
static QueueHandle_t xBtnEventQueue  = NULL; // Button event queue

/* ===== RMT Handles ===== */
static rmt_channel_handle_t rmt_tx_ch   = NULL;
static rmt_encoder_handle_t rmt_encoder = NULL;
static rmt_transmit_config_t rmt_tx_cfg = { .loop_count = 0 };
static rmt_copy_encoder_config_t rmt_copy_cfg = {};

```

5.2 ws2812_init() — WS2812 Initialization

Drive WS2812 via RMT peripheral. The RMT (Remote Control) module can generate precise timing pulses, suitable for driving digital LEDs requiring nanosecond-level timing.

```

static void ws2812_init(void)
{
    rmt_tx_channel_config_t tx_ch_cfg = {
        .gpio_num = RGB_LED_GPIO,          // GPIO 48
        .clk_src  = RMT_CLK_SRC_DEFAULT,
        .resolution_hz = RMT_RESOLUTION_HZ,
        .mem_block_symbols = 64,
        .trans_queue_depth = 4,
    };
    ESP_ERROR_CHECK(rmt_new_tx_channel(&tx_ch_cfg, &rmt_tx_ch));
    ESP_ERROR_CHECK(rmt_new_copy_encoder(&rmt_copy_cfg, &rmt_encoder));
    ESP_ERROR_CHECK(rmt_enable(rmt_tx_ch));

    ws2812_set_color(LED_COLOR_OFF);
}

```

5.3 ws2812_write_rgb() — Write GRB Color

WS2812 receives data in **GRB byte order** (Green → Red → Blue), each byte **MSB first**. The following function manually encodes 24-bit color data into RMT symbol sequences:

```
static void ws2812_write_rgb(uint8_t r, uint8_t g, uint8_t b)
{
    // 24 bits * 2 RMT symbols per bit + 1 reset symbol
    rmt_symbol_word_t symbols[24 * 2 + 1];
    int idx = 0;
    uint32_t grb = ((uint32_t)g << 16) | ((uint32_t)r << 8) | (uint32_t)b;

    // Encode 24 bits MSB-first
    for (int bit = 23; bit >= 0; bit--) {
        symbols[idx].val = 0;
        if (grb & (1 << bit)) {
            // 1-bit: 0.8us HIGH + 0.5us LOW
            symbols[idx].duration0 = WS2812_T1H;
            symbols[idx].level0 = 1;
            symbols[idx].duration1 = WS2812_T1L;
            symbols[idx].level1 = 0;
        } else {
            // 0-bit: 0.4us HIGH + 0.9us LOW
            symbols[idx].duration0 = WS2812_T0H;
            symbols[idx].level0 = 1;
            symbols[idx].duration1 = WS2812_T0L;
            symbols[idx].level1 = 0;
        }
        idx++;
    }
    // Reset code (>50us LOW)
    symbols[idx].val = 0;
    symbols[idx].duration0 = WS2812_RESET_TICKS;
    symbols[idx].level0 = 0;
    symbols[idx].duration1 = 0;
    symbols[idx].level1 = 0;
    idx++;

    ESP_ERROR_CHECK(rmt_transmit(rmt_tx_ch, rmt_encoder,
                                symbols, idx * sizeof(rmt_symbol_word_t),
                                &rmt_tx_cfg));
    ESP_ERROR_CHECK(rmt_tx_wait_all_done(rmt_tx_ch, pdMS_TO_TICKS(100)));
}
```

5.4 vButtonTask() — Button Detection Task

Poll GPIO0, detect short/long press after debouncing, send events to CtrlTask via queue. This is a typical state machine pattern — detecting edges by comparing `prev_state` and `cur_state`:

```
static void vButtonTask(void *pvParameters)
{
    gpio_config_t io_cfg = {
```

```

        .intr_type    = GPIO_INTR_DISABLE,
        .mode         = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << BTN_GPIO),
        .pull_up_en   = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
    };
    gpio_config(&io_cfg);

    int press_duration_ms = 0;
    bool prev_state = true; // HIGH = not pressed (pull-up)

    while (1) {
        bool cur_state = gpio_get_level(BTN_GPIO);

        if (prev_state == true && cur_state == false) {
            // Falling edge: button pressed
            press_duration_ms = 0;
        } else if (prev_state == false && cur_state == false) {
            // Held down
            press_duration_ms += BTN_POLL_PERIOD_MS;
        } else if (prev_state == false && cur_state == true) {
            // Rising edge: button released
            if (press_duration_ms >= BTN_DEBOUNCE_MS) {
                btn_event_t evt;
                if (press_duration_ms >= BTN_LONG_PRESS_MS) {
                    evt = BTN_EVT_LONG;
                    ESP_LOGI(TAG, ">> Long press detected (%dms)", press_duration_ms);
                } else {
                    evt = BTN_EVT_SHORT;
                    ESP_LOGI(TAG, ">> Short press detected (%dms)", press_duration_ms);
                }
                xQueueSend(xBtnEventQueue, &evt, 0);
            }
            press_duration_ms = 0;
        }

        prev_state = cur_state;
        vTaskDelay(pdMS_TO_TICKS(BTN_POLL_PERIOD_MS));
    }
}

```

5.5 vLedTask() — LED Task

Runs independently on Core 1, cycles through four colors. Short press suspends, long press deletes. When suspended, LED stays at current color:

```

static void vLedTask(void *pvParameters)
{
    ESP_LOGI(TAG, "LED Task started on core %d", xPortGetCoreID());
    led_color_t colors[] = { LED_COLOR_RED, LED_COLOR_GREEN,
                             LED_COLOR_BLUE, LED_COLOR_WHITE };
    int idx = 0;

```



```

while (1) {
    ws2812_set_color(colors[idx]);
    ESP_LOGI(TAG, "LED: %d | Core: %d | Priority: %d",
               idx, xPortGetCoreID(), uxTaskPriorityGet(NULL));

    idx = (idx + 1) % 4;
    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

5.6 vMainControlTask() — Main Control Task

Receives button events from queue, calls `vTaskSuspend` / `vTaskResume` / `vTaskDelete` to control LedTask. This is the core scheduling logic of the entire project:

```

static void vMainControlTask(void *pvParameters)
{
    ESP_LOGI(TAG, "Tasks ready. BOOT (GPIO 0): short=suspend/resume, long=delete LED task");

    xTaskCreatePinnedToCore(vLedTask, "Led_Task", 3072, NULL, 2,
                           &xLedTaskHandle, 1);

    xTaskCreatePinnedToCore(vCountTask, "Count_Task", 2048, NULL, 2,
                           &xCountTaskHandle, 0);

    bool led_suspended = false;
    btn_event_t evt;

    while (1) {
        if (xQueueReceive(xBtnEventQueue, &evt, portMAX_DELAY) == pdTRUE) {

            if (evt == BTN_EVT_SHORT) {
                if (xLedTaskHandle == NULL) {
                    ESP_LOGW(TAG, "LED task already deleted");
                    continue;
                }

                if (led_suspended) {
                    ESP_LOGI(TAG, "===== Resume LED Task =====");
                    vTaskResume(xLedTaskHandle);
                    led_suspended = false;
                } else {
                    ESP_LOGI(TAG, "===== Suspend LED Task =====");
                    vTaskSuspend(xLedTaskHandle);
                    led_suspended = true;
                }
            }

            } else if (evt == BTN_EVT_LONG) {
                if (xLedTaskHandle != NULL) {
                    ESP_LOGI(TAG, "===== Delete LED Task =====");
                    if (!led_suspended) {

```

```

        vTaskResume(xLedTaskHandle);
        led_suspended = false;
    }
    vTaskDelete(xLedTaskHandle);
    xLedTaskHandle = NULL;
    ws2812_set_color(LED_COLOR_OFF);
    ESP_LOGI(TAG, "LED Task deleted");
} else {
    ESP_LOGW(TAG, "LED task already deleted");
}
}
}
}
}
}
}

```

Why Resume before Delete? FreeRTOS's `vTaskDelete` cannot delete tasks in Suspended state. Must resume first, then delete.

5.7 app_main() — Main Entry

`app_main` is the entry point of the entire program, responsible for initializing hardware, creating queues, and launching three FreeRTOS tasks:

```

void app_main(void)
{
    ESP_LOGI(TAG, "==== RTOS Task Control via Button =====");

    ws2812_init();

    xBtnEventQueue = xQueueCreate(5, sizeof(btn_event_t));

    xTaskCreatePinnedToCore(vButtonTask, "Btn_Task", 2048, NULL, 1, NULL, 0);

    xTaskCreatePinnedToCore(vMainControlTask, "Ctrl_Task", 3072, NULL, 1, NULL, 0);
}

```

6. Operation Flow

6.1 Compilation and Flashing Flow

For detailed flashing process, see: **1. Before Development** → **3. Build and Flash**

6.2 Normal Operation

1. After flashing, power on, WS2812 cycles colors (red→green→blue→white, switching every 1 second)
2. **Short press** BOOT button (hold < 1s) → LED stays at current color (task suspended)
3. **Short press again** → LED resumes color cycling (task resumed)
4. **Long press** BOOT button (hold ≥ 1s) → LED turns off, task deleted
5. CountTask always counts independently on Core 0, unaffected by LedTask state

Log field meanings:

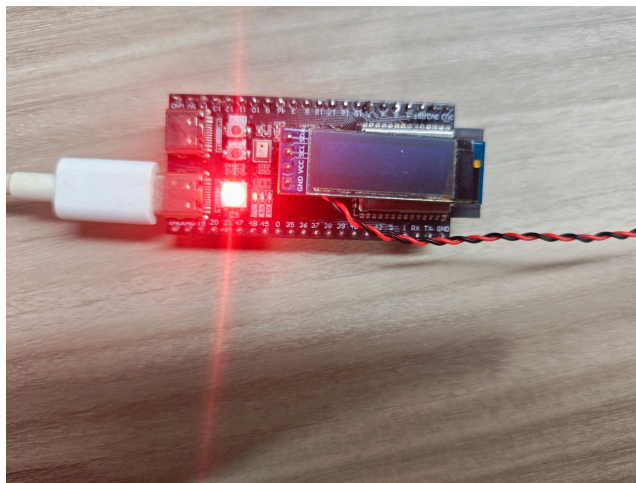
Field	Example	Meaning
I	I	INFO level log
FREERTOS_TASK	FREERTOS_TASK	Log tag
LED: 2	2	Current color index (0=red, 1=green, 2=blue, 3=white)
Core: 1	1	Task running on Core 1
Priority: 2	2	Current task priority
>> Short press detected	—	Short press detected
>> Long press detected	—	Long press detected

The following is typical serial log during normal operation, showing the complete process of task creation, LED cycling, suspend/resume, and deletion:

```
I (281) FREERTOS_TASK: ===== RTOS Task Control via Button =====
I (281) main_task: Returned from app_main()
I (281) FREERTOS_TASK: Tasks ready. BOOT (GPIO 0): short=suspend/resume, long=delete LED
task
I (291) FREERTOS_TASK: LED Task started on core 1
I (291) FREERTOS_TASK: Count: 1 | Stack free: 1504 words
I (291) FREERTOS_TASK: LED: 0 | Core: 1 | Priority: 2
I (1301) FREERTOS_TASK: Count: 2 | Stack free: 816 words
I (1301) FREERTOS_TASK: LED: 1 | Core: 1 | Priority: 2
I (2301) FREERTOS_TASK: Count: 3 | Stack free: 816 words
I (2301) FREERTOS_TASK: LED: 2 | Core: 1 | Priority: 2
I (3301) FREERTOS_TASK: Count: 4 | Stack free: 816 words
I (3301) FREERTOS_TASK: LED: 3 | Core: 1 | Priority: 2
I (3671) FREERTOS_TASK: >> Short press detected (180ms)
I (3671) FREERTOS_TASK: ===== Suspend LED Task =====
I (4301) FREERTOS_TASK: Count: 5 | Stack free: 816 words
I (5301) FREERTOS_TASK: Count: 6 | Stack free: 816 words
I (6301) FREERTOS_TASK: Count: 7 | Stack free: 816 words
I (7301) FREERTOS_TASK: Count: 8 | Stack free: 816 words
I (7691) FREERTOS_TASK: >> Short press detected (200ms)
I (7691) FREERTOS_TASK: ===== Resume LED Task =====
I (7691) FREERTOS_TASK: LED: 0 | Core: 1 | Priority: 2
I (8301) FREERTOS_TASK: Count: 9 | Stack free: 816 words
I (8691) FREERTOS_TASK: LED: 1 | Core: 1 | Priority: 2
I (9301) FREERTOS_TASK: Count: 10 | Stack free: 816 words
I (9691) FREERTOS_TASK: LED: 2 | Core: 1 | Priority: 2
I (10301) FREERTOS_TASK: Count: 11 | Stack free: 816 words
I (10691) FREERTOS_TASK: LED: 3 | Core: 1 | Priority: 2
I (11301) FREERTOS_TASK: Count: 12 | Stack free: 816 words
I (11691) FREERTOS_TASK: LED: 0 | Core: 1 | Priority: 2
I (12301) FREERTOS_TASK: Count: 13 | Stack free: 816 words
I (12691) FREERTOS_TASK: LED: 1 | Core: 1 | Priority: 2
```

```
I (13301) FREERTOS_TASK: Count: 14 | Stack free: 816 words
I (13691) FREERTOS_TASK: LED: 2 | Core: 1 | Priority: 2
I (14301) FREERTOS_TASK: Count: 15 | Stack free: 816 words
I (14691) FREERTOS_TASK: LED: 3 | Core: 1 | Priority: 2
I (15231) FREERTOS_TASK: >> Long press detected (3150ms)
I (15231) FREERTOS_TASK: ===== Delete LED Task =====
I (15231) FREERTOS_TASK: LED Task deleted
I (15301) FREERTOS_TASK: Count: 16 | Stack free: 816 words
```

```
I (281) FREERTOS_TASK: ===== RTOS Task Control via Button =====
I (281) main_task: Returned from app_main()
I (281) FREERTOS_TASK: Tasks ready. BOOT (GPIO 0): short=suspend/resume, long=delete LED task
I (291) FREERTOS_TASK: LED Task started on core 1
I (291) FREERTOS_TASK: Count: 1 | Stack free: 1504 words
I (291) FREERTOS_TASK: LED: 0 | Core: 1 | Priority: 2
I (1301) FREERTOS_TASK: Count: 2 | Stack free: 816 words
I (1301) FREERTOS_TASK: LED: 1 | Core: 1 | Priority: 2
I (2301) FREERTOS_TASK: Count: 3 | Stack free: 816 words
I (2301) FREERTOS_TASK: LED: 2 | Core: 1 | Priority: 2
I (3301) FREERTOS_TASK: Count: 4 | Stack free: 816 words
I (3301) FREERTOS_TASK: LED: 3 | Core: 1 | Priority: 2
I (3671) FREERTOS_TASK: >> Short press detected (180ms)
I (3671) FREERTOS_TASK: ===== Suspend LED Task =====
I (4301) FREERTOS_TASK: Count: 5 | Stack free: 816 words
I (5301) FREERTOS_TASK: Count: 6 | Stack free: 816 words
I (6301) FREERTOS_TASK: Count: 7 | Stack free: 816 words
I (7301) FREERTOS_TASK: Count: 8 | Stack free: 816 words
I (7691) FREERTOS_TASK: >> Short press detected (200ms)
I (7691) FREERTOS_TASK: ===== Resume LED Task =====
I (7691) FREERTOS_TASK: LED: 0 | Core: 1 | Priority: 2
I (8301) FREERTOS_TASK: Count: 9 | Stack free: 816 words
I (8691) FREERTOS_TASK: LED: 1 | Core: 1 | Priority: 2
I (9301) FREERTOS_TASK: Count: 10 | Stack free: 816 words
I (9691) FREERTOS_TASK: LED: 2 | Core: 1 | Priority: 2
I (10301) FREERTOS_TASK: Count: 11 | Stack free: 816 words
I (10691) FREERTOS_TASK: LED: 3 | Core: 1 | Priority: 2
I (11301) FREERTOS_TASK: Count: 12 | Stack free: 816 words
I (11691) FREERTOS_TASK: LED: 0 | Core: 1 | Priority: 2
I (12301) FREERTOS_TASK: Count: 13 | Stack free: 816 words
I (12691) FREERTOS_TASK: LED: 1 | Core: 1 | Priority: 2
I (13301) FREERTOS_TASK: Count: 14 | Stack free: 816 words
I (13691) FREERTOS_TASK: LED: 2 | Core: 1 | Priority: 2
I (14301) FREERTOS_TASK: Count: 15 | Stack free: 816 words
I (14691) FREERTOS_TASK: LED: 3 | Core: 1 | Priority: 2
I (15231) FREERTOS_TASK: >> Long press detected (3150ms)
I (15231) FREERTOS_TASK: ===== Delete LED Task =====
I (15231) FREERTOS_TASK: LED Task deleted
I (15301) FREERTOS_TASK: Count: 16 | Stack free: 816 words
```



7. FAQ

Symptom	Cause	Solution
<code>vTaskDelete</code> ineffective, task still running	Target task is Suspended	Resume first with <code>vTaskResume</code> then <code>vTaskDelete</code>
LED color abnormal (e.g., blue becomes green)	WS2812 GRB byte order error	Confirm write order is Green → Red → Blue
LED not lighting	RMT initialization failed / GPIO config error	Check GPIO48 wiring, confirm RMT resolution
Button insensitive	Debounce time too short	Increase <code>BTN_DEBOUNCE_MS</code> (recommended 30~50ms)
Task creation failed (returns -1)	Stack too small / insufficient memory	Increase <code>usStackDepth</code> (recommended ≥ 2048)
Long press misidentified as multiple short presses	No debounce on release	Add 50ms delay on button release
<code>LED task already deleted</code>	Handle not set to NULL after deletion / duplicate operation	Set <code>xLedTaskHandle = NULL</code> after deletion